

Autonomous Polyglot Debugging & Repository Intelligence Engine

A comprehensive technical deep-dive into Fixie's 6-agent LangGraph orchestration pipeline, Server-Sent Events streaming engine, MySQL session memory layer, and Obsidian-dark React 18 Studio frontend.

LangGraph & LangChain

Google Gemini 1.5 Flash

FastAPI & Python 3.10+

MySQL 8.0 Persistence

React 18 & Vite

Polyglot: C++, JS/TS, Python, Java, Go, Rust

Dual Execution: Snippet & Full-Repo

TABLE OF CONTENTS

1. Executive Summary & Core Value Proposition	Section 1
2. High-Level System Architecture & End-to-End Data Flow	Section 2
3. The 6-Agent LangGraph Multi-Agent Pipeline	Section 3
4. Core Services & Infrastructure Subsystem	Section 4
5. FastAPI Backend Server & Endpoint Specifications	Section 5
6. MySQL Database Subsystem & Session Memory	Section 6
7. Frontend Studio Architecture & Component Hierarchy	Section 7
8. Testing Fixtures, Sample Repositories & Operational Runbook	Section 8

1 Executive Summary & Core Value Proposition

Fixie is an autonomous, multi-agent AI debugging studio designed to eliminate the manual, error-prone cycle of diagnosing bugs, understanding legacy logic, writing patches, and constructing verification tests across heterogeneous programming languages and complex repository structures.

SNIPPET MODE Focused Algorithmic & File Debugging

Designed for LeetCode algorithms, single-file scripts, competitive programming, and isolated functions. Evaluates syntax/compilation rules, intent flow, edge cases, and produces executable test harnesses.

REPOSITORY MODE Full-Project Cross-File Auditing

Ingests entire multi-file codebases (MERN web apps, C++ CMake engines, microservices). Maps inter-file dependencies, API contracts, scouts bugs across modules, and provides cross-file context to the repair engine.

Key Architectural Capabilities:

- **Multi-Language Polyglot Support:** Natively supports C++, Python, JavaScript/React, TypeScript, Java, Go, Rust, and systems languages with specialized compiler AST and memory safety heuristics.
- **Zero-Simulation Real-Time Streaming:** Built on genuine Server-Sent Events (SSE). Nodes push live execution state chunks as each agent in the LangGraph finishes work, giving real-time visual pipeline feedback without synthetic delays.
- **Client-Side & Server-Side Dual Ingestion:** Projects can be scanned locally on the server file system, cloned from remote Git repositories, or parsed client-side in the browser using the HTML5 `webkitdirectory` and `DataTransfer` APIs without server file upload overhead.
- **Dual Verification Harnesses:** Produces both executable programmatic test suites (for LeetCode/CLI) and interactive browser verification runbooks (for MERN/frontend applications).
- **Persistent Diagnostic Memory:** Retains historical debugging runs, code revisions, and bug classifications inside a MySQL 8.0 relational database with 1-click historical reload.

2

The system consists of three interconnected layers: the **React 18 Studio Frontend**, the **FastAPI Orchestration Gateway**, and the **LangGraph Multi-Agent Engine** backed by the **Google Gemini API** and **MySQL Session Storage**.



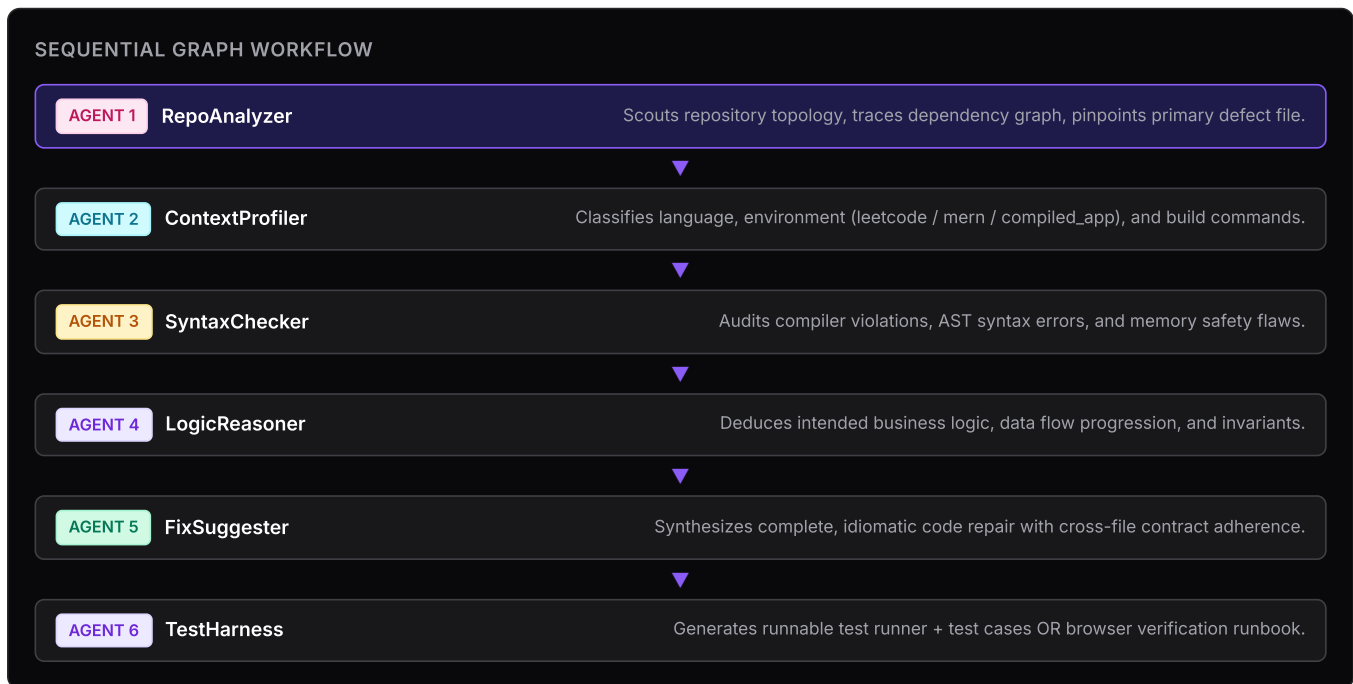
Data Flow Lifecycle:

1. **Ingestion:** The user selects Snippet Mode (pastes code or selects preset) or Repo Mode (specifies local path, drops folder, or clicks sample repo).
2. **Dispatch:** Frontend issues an HTTP `POST /api/debug/stream` request with the payload (code, mode, repo manifest, focus file).
3. **Graph Execution:** FastAPI dispatches an asynchronous worker thread invoking `stream_fixie_graph()` on the LangGraph state machine.
4. **Real-Time SSE Yields:** As each agent node finishes, it yields a `node_complete` event over the SSE connection containing its structured telemetry.
5. **UI Animation & Rendering:** Frontend's `EventSource` / stream reader updates the pipeline step indicator from *running* to *completed* and activates the subsequent agent.
6. **Persistence & Termination:** Upon graph termination, the session is saved to MySQL, a `complete` event with the `session_id` is dispatched, and the diagnostic results panel renders all solutions, tests, and traces.

3 The 6-Agent LangGraph Multi-Agent Pipeline

The core reasoning brain of Fixie resides in `core/langgraph_runner.py` and six specialized agents located in the `agents/` directory. Agents communicate through a shared state container defined by the `FixieState` TypedDict.

FixieState Schema: Holds `mode`, `repo_path`, `repo_manifest`, `repo_analysis`, `focus_file`, `code`, `language`, `context_profile`, `syntax_report`, `logic`, `fix`, and `test_suite`.



Deep-Dive Agent Specifications

1. RepoAnalyzer (`agents/repo_analyzer.py`)

- Role:** Principal Software Architect & Full-Repository Code Auditor.
- Inputs:** Repository manifest containing filtered file tree, build configs (`package.json`, `CMakeLists.txt`), and source file contents.
- Outputs:** JSON containing `stack_summary`, `architecture_type`, `dependency_graph` (caller/callee module mapping), `build_and_test_commands`, `detected_issues` list with file paths and severity, `primary_target_file`, and `cross_file_context`.
- Mechanism:** In Snippet Mode, passes through instantly. In Repo Mode, isolates the primary buggy file and loads its contents into `state["code"]` for downstream agents.

2. ContextProfiler (`agents/context_profiler.py`)

- Role:** Senior Software Architect & Build Environment Classifier.
- Inputs:** Source code and optional user language hint (or "auto").
- Outputs:** `language`, `display_language`, `environment_type` (`leetcode`, `mern`, `compiled_app`, `backend_api`, `script`), `build_system`, and `test_strategy`.
- Mechanism:** Employs few-shot prompt heuristics and regex fallback to detect idioms (e.g. `vector<int>& nums` → C++ LeetCode, `import React, { useState }` → React MERN).

3. SyntaxChecker (`agents/syntax_checker.py`)

- Role:** Expert Compiler, Linter, and Memory Safety Diagnostician.
- Inputs:** Source code, detected language, and context profile (including cross-file context).
- Outputs:** `bug_explanation`, `line_number`, and `severity` (`high`, `medium`, `low`, `none`).
- Mechanism:** Detects language-specific compiler traps: out-of-bounds array access in C++, unhandled Promise awaits in Express, infinite re-render loops in React hooks, and division-by-zero in Python.

4. LogicReasoner (`agents/logic_reasoner.py`)

- Role:** Algorithmic Logic Explainer & Program Intent Analyst.

- **Inputs:** Source code, language, and context profile.
- **Outputs:** Structured Markdown breakdown covering Intended Logic, Expected Data Flow, and Required Constraints/Invariants.
- **Mechanism:** Reconstructs what the author attempted to achieve before the defect was introduced, providing cognitive grounding for the fix generator.

5. FixSuggester (`agents/fix_suggester.py`)

- **Role:** Master Software Engineer & Code Synthesis Engine.
- **Inputs:** Source code, bug report, logic explanation, language, and cross-file contract context.
- **Outputs:** `explanation`, complete corrected `fix` code, and `confidence` score (0.0 to 1.0).
- **Mechanism:** Generates the full, drop-in replacement function or module (never partial pseudo-diffs) preserving conventions and variable naming while honoring cross-file types and endpoints.

6. TestHarness (`agents/test_harness.py`)

- **Role:** Senior QA Automation Engineer & Verification Architect.
- **Inputs:** Original code, suggested fix, context profile, and logic breakdown.
- **Outputs:** Dual-strategy JSON:
 - **Algorithmic Mode:** 3-4 diverse test cases (normal, edge, boundary) + complete runnable test script in the target language.
 - **Web / MERN Mode:** Browser runbook with dev server startup command, target URL, step-by-step reproduction/verification guide, expected UI visuals, and DevTools console checklist.
 - **CLI / Systems Mode:** Compiler invocation command and execution flags.

4 Core Services & Infrastructure Subsystem

The `core/` directory contains foundational utilities that decouple external APIs, file system scanning, and LLM providers from high-level agent logic.

CORE/GEMINI_INTERFACE.PY

Gemini Client & REST Fallback:

- Attempts to query via official `google-genai` SDK using `genai.Client(api_key=...)`.
- If the SDK is unavailable or encounters environment issues, automatically falls back to direct HTTP REST calls via `httpx.Client` to `https://generativelanguage.googleapis.com/v1beta/models/{model}:generateContent`.
- Validates API key status through `is_gemini_configured()`.

CORE/REPO_SCANNER.PY

Repository File Tree & Git Ingestion:

- Recursively traverses directories while skipping noise directories (`node_modules`, `.git`, `__pycache__`, `dist`, `build`, `venv`).
- Ignores binary file extensions (`.exe`, `.so`, `.png`, `.zip`, `.pdf`).
- Recognizes manifest files (`package.json`, `CMakeLists.txt`, `requirements.txt`, `Cargo.toml`).
- Supports remote Git cloning via `clone_and_scan_repo(url)`.

CORE/LANGGRAPH_RUNNER.PY

Graph Construction & Dual Execution Handlers:

```
def build_fixie_graph():
    builder = StateGraph(FixieState)
    builder.add_node("RepoAnalyzer", repo_node)
    builder.add_node("ContextProfiler", context_node)
    builder.add_node("SyntaxChecker", syntax_node)
    builder.add_node("LogicReasoner", logic_node)
    builder.add_node("FixSuggester", fix_node)
    builder.add_node("TestHarness", test_node)

    builder.set_entry_point("RepoAnalyzer")
    builder.add_edge("RepoAnalyzer", "ContextProfiler")
    builder.add_edge("ContextProfiler", "SyntaxChecker")
    builder.add_edge("SyntaxChecker", "LogicReasoner")
    builder.add_edge("LogicReasoner", "FixSuggester")
    builder.add_edge("FixSuggester", "TestHarness")
    builder.add_edge("TestHarness", "__end__")
    return builder.compile()
```

Exposes `run_fixie_graph()` for single-call synchronous debugging and `stream_fixie_graph()` which yields intermediate chunk objects containing each node's output as it executes.

5 FastAPI Backend Server & Endpoint Specifications

The backend server is implemented in `app.py` using **FastAPI** and **Uvicorn**. It connects the web client to the agent pipeline, serves the React production bundle, and provides diagnostic REST/SSE endpoints.

Method	Endpoint	Description	Payload / Parameters	Response Structure
GET	/	Serves compiled React frontend or static fallback.	None	HTML document
GET	/api/health	Diagnostic check for MySQL server and Gemini API key.	None	<code>{ status, mysql: { connected, message }, gemini: { available, message } }</code>
GET	/api/examples	Multi-language preset buggy snippets for 1-click testing.	None	<code>{ examples: [{ title, language, description, code }] }</code>
GET	/api/sessions	Retrieves recent debugging sessions from MySQL database.	<code>limit: int = 20</code>	<code>{ sessions: [DebuggingSession] }</code>
POST	/api/repo/scan	Scans a local directory path or clones and scans a Git URL.	<code>{ repo_path: str }</code>	<code>{ success: bool, repo: RepoManifest }</code>
GET	/api/repo/examples	Returns pre-configured sample repositories (MERN, C++ CMake).	None	<code>{ sample_repos: [{ name, path, category, description }] }</code>
POST	/api/debug	Runs complete LangGraph pipeline synchronously; saves session.	<code>DebugRequest (code, language, mode, repo_path, repo_manifest)</code>	<code>DebugResponse (syntax_report, logic, fix, test_suite, session_id)</code>
POST	/api/debug/stream	Streams agent node execution in real time via Server-Sent Events.	<code>DebugRequest</code>	<code>text/event-stream (data: { type, node, data })</code>

Server-Sent Events (SSE) Protocol Details

When calling `/api/debug/stream`, the client receives a live event stream with three message types:

- `{ "type": "init", "message": "..."}` : Handshake notification confirming pipeline startup.
- `{ "type": "node_complete", "node": "<AgentName>", "data": {...}}` : Dispatched immediately as each agent finishes, delivering incremental results.
- `{ "type": "complete", "session_id": 42, "db_saved": true, "state": {...}}` : Final termination event delivering session persistence ID and full state.
- `{ "type": "error", "message": "..."}` : Dispatched upon unexpected execution failure.

6 MySQL Database Subsystem & Session Memory

Session memory is managed within the `database/` package using **PyMySQL**, providing persistent storage for debug runs, language metrics, and error histories.

DATABASE/MODELS.PY Schema Definition

```
CREATE TABLE IF NOT EXISTS debugging_sessions (  
    id INT AUTO_INCREMENT PRIMARY KEY,  
    language VARCHAR(50) NOT NULL DEFAULT 'python',  
    original_code LONGTEXT NOT NULL,  
    error_description TEXT,  
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,  
    INDEX idx_language (language),  
    INDEX idx_created_at (created_at)  
) ENGINE=InnoDB  
    DEFAULT CHARSET=utf8mb4  
    COLLATE=utf8mb4_unicode_ci;
```

DATABASE/SERVICE.PY Service Layer Methods

- `check_connection() → (bool, str)`: Validates MySQL server availability and checks if the database/tables exist.
- `init_db() → bool`: Automatically issues `CREATE DATABASE IF NOT EXISTS` and executes DDL table creation.
- `create_session(code, language, error) → DebuggingSession`: Records a completed debug run.
- `list_sessions(limit) → List[DebuggingSession]`: Fetches recent sessions ordered by timestamp descending.

Database Diagnostics Tool: `verify_db.py`

A dedicated CLI tool provides immediate verification, automated migrations, and CRUD roundtrip checks:

```
# Check MySQL server reachability and schema status  
python verify_db.py  
  
# Initialize database schema and required tables  
python verify_db.py --init  
  
# Run end-to-end diagnostics: ping, initialization, and CRUD roundtrip  
python verify_db.py --test-all
```

7 Frontend Studio Architecture & Component Hierarchy

The frontend is a single-page application built with **React 18**, **Vite**, custom **Vanilla CSS** (Obsidian dark aesthetic), and **Lucide React** icons. It is designed without external CSS frameworks to ensure maximum performance and pixel-perfect design control.

```
FRONTEND COMPONENT HIERARCHY

<App /> (Root State Machine, SSE Stream Consumer, Toast Dispatcher)
|-- <Navbar /> (Mode Toggle, Health Diagnostics Pills, History Drawer Trigger)
|-- <main className="studio-layout">
|   |-- [Left Column]:
|   |   |-- (Snippet Mode) <CodeEditor /> (Gutter Line Numbers, Presets, Run/Clear)
|   |   |-- (Repo Mode) <RepoExplorer /> (Folder Dropzone, Tree, Active Viewer)
|   |-- [Right Column]:
|   |   |-- <AgentPipeline /> (6-Agent Status Timeline: idle/running/completed/error)
|   |   |-- <ResultsPanel /> (Tabbed Diagnostics Studio)
|   |       |-- Tab: Repo Architecture (Topology, Dependency Graph, Build Cmds)
|   |       |-- Tab: Solution & Fix (Corrected Code, Copy Button, Confidence %)
|   |       |-- Tab: Verification & Tests (Runnable Test Script / Browser Runbook)
|   |       |-- Tab: Bug Analysis (Severity Badge, Line Number, Explanation)
|   |       |-- Tab: Logic Breakdown (Algorithmic Intent, Data Flow, Invariants)
|   |       |-- Tab: Raw Trace (Full JSON telemetry payload)
|   |-- <HistoryDrawer /> (Slide-out past debugging sessions panel with 1-click reload)
|-- <Toast /> (Floating non-blocking notification alerts)
```

Detailed Component Breakdown

Component File	Primary Responsibility	Key Features & Interactions
<code>Navbar.jsx</code>	Top navigation and system telemetry bar.	Mode switcher (Snippet vs Repo), real-time MySQL and Gemini health status pills with tooltips, session count badge, drawer open trigger.
<code>CodeEditor.jsx</code>	Syntax-aware code editor for snippet mode.	Dynamic line number gutters synchronized with scrolling, language selector dropdown, preset code examples, run debugger CTA, clear button.
<code>RepoExplorer.jsx</code>	Full-repository exploration workspace.	Drag-and-drop folder dropzone, native folder picker dialog, sample repository 1-click loaders, file search filter, tree viewer with bug badges, integrated code preview.
<code>AgentPipeline.jsx</code>	Visual execution timeline of the 6 agents.	Displays status icons (clock, spinner, check, alert) and pulsating progress indicators as agents execute in real time.
<code>ResultsPanel.jsx</code>	Multi-tab diagnostics and solution view.	Displays confidence gauge, severity badges, syntax report, full fixed code block with clipboard copy, runnable test runner script, browser verification steps, and raw telemetry.
<code>HistoryDrawer.jsx</code>	Slide-out past debugging sessions drawer.	Lists historical sessions retrieved from MySQL with timestamp and language; clicking a session instantly restores code into the editor.
<code>folderParser.js</code>	Client-side folder ingestion engine.	Uses HTML5 <code>webkitdirectory</code> and <code>DataTransferItem.webkitGetAsEntry</code> to recursively read local directories in the browser, filtering noise and producing manifest JSON without uploading files.

8 Testing Fixtures, Sample Repositories & Operational Runbook

Built-In Testing Fixtures & Repositories

Fixie includes realistic multi-file test repositories under `examples/sample_repos/` designed to validate cross-file agent reasoning:

MERN STACK HARDWARE STORE

`examples/sample_repos/mern_store`

- **Frontend:** React (Vite) with `ProductList.jsx` and `Navbar.jsx`. Contains an infinite `useEffect` fetch loop due to missing dependency array.
- **Backend:** Express.js with `server.js` and `routes/products.js`. Contains an API contract mismatch (returning `item_name` instead of expected `title`).
- **Agent Validation:** RepoAnalyzer traces the frontend-backend contract mismatch across files and fixes both the loop and interface.

C++ HIGH PERFORMANCE ENGINE

`examples/sample_repos/cpp_engine`

- **Structure:** CMake build system with `CMakeLists.txt`, `src/engine.h`, `src/engine.cpp`, and `src/main.cpp`.
- **Bugs:** Contains an off-by-one buffer overflow in array processing and an unreleased dynamic memory allocation (memory leak).
- **Agent Validation:** RepoAnalyzer identifies CMake targets, SyntaxChecker detects memory safety hazards, and TestHarness produces compiler flags (`g++ -Wall -fsanitize=address`).

Quick Start & Operational Runbook

```
# 1. Clone repository and set up virtual environment
git clone <repo-url>
cd DebuggingAI
python -m venv fixie
fixie\Scripts\activate

# 2. Install Python dependencies
pip install -r requirements.txt

# 3. Configure Environment (.env)
GEMINI_API_KEY=your_gemini_api_key_here
GEMINI_MODEL=gemini-1.5-flash
DB_HOST=localhost
DB_PORT=3306
DB_USER=root
DB_PASSWORD=your_mysql_password
DB_NAME=fixie_db

# 4. Initialize Database
python verify_db.py --init

# 5. Build and Launch Frontend
cd frontend
npm install
npm run build
cd ..

# 6. Start FastAPI Application Server
uvicorn app:app --host 0.0.0.0 --port 8000 --reload
```

Operational Tip: Navigate to `http://localhost:8000` in your browser to access the unified production studio. For frontend hot-reloading during development, run `npm run dev` inside `frontend/` and access `http://localhost:5173` (CORS is pre-configured).